

Yesterday 3:21 PM

What is a record?
Why were records introduced?
Record vs normal class.
Can records be mutable?
Can records extend another class?
Can records implement interfaces?
What are sealed classes?
Why use sealed classes?
Sealed class vs final class.
What is pattern matching?
Pattern matching with instanceof.
[Show more](#) ✓

Absolutely. For **5 years of Java experience**, you should answer these with both the definition and the practical reason behind the feature.

1. What is a Record?

A **record** is a special type of class introduced in **Java 16** for creating immutable data-carrier objects with less boilerplate.

 **Java**



```
public record Employee(Long id, String name, double salary) {  
}
```

Java automatically provides:

- `private final` fields
- Constructor
- Accessor methods: `id()`, `name()`, `salary()`
- `equals()`
- `hashCode()`

- `toString()`

Usage:

</> Java



```
Employee e = new Employee(101L, "Manoj", 50000);

System.out.println(e.name());
```

Interview answer

"Record is a concise way to model immutable data. It is useful for DTOs, API responses, configuration objects, and other objects whose primary purpose is carrying data."

2. Why were Records introduced?

Before records, a simple DTO required a lot of boilerplate:

</> Java



```
class Employee {

    private final Long id;
    private final String name;

    public Employee(Long id, String name) {
        this.id = id;
        this.name = name;
    }

    public Long getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    // equals()
    // hashCode()
    // toString()
}
```

With a record:

</> Java

record Employee(Long id, String name) {}

So records were introduced primarily to:

- Reduce boilerplate
- Clearly express data-carrier intent
- Provide value-based equals/hashCode
- Encourage immutable data modeling

3. Record vs Normal Class

Record	Normal Class
Designed primarily for data	General-purpose
Components are final	Fields can be mutable
Automatically gets constructor	You write constructor
Automatically gets accessors	You write getters if required
Auto equals() / hashCode()	Usually manually implemented
Cannot extend another class	Can extend a class
Can implement interfaces	Can implement interfaces
More concise	More flexible

Example:

</> Java

record Employee(Long id, String name) {}

VS

</> Java



```
class Employee {  
    private final Long id;  
    private final String name;  
  
    // constructor/getters>equals/hashCode...  
}
```

Important interview point

A record is **not** simply a replacement for every class.

If you need:

- inheritance from another class
- mutable state
- complex object lifecycle
- framework-specific proxy behavior

a normal class may be more appropriate.

4. Can Records be Mutable?

A record's **components** are **final**, so you cannot reassign them.

</> Java



```
record Employee(String name) {}  
  
Employee e = new Employee("Manoj");  
  
// e.name = "Rahul"; // ❌
```

But there is an important nuance.

A record itself is **shallowly immutable**, not necessarily deeply immutable.

</> Java



```
record Employee(List<String> skills) {}
```

This reference is final:

</> Java



```
employee.skills = anotherList; // ❌
```

But the list itself could still be modified:

</> Java



```
employee.skills().add("Java");
```

if the supplied list is mutable.

Interview answer

"Records provide shallow immutability because their components are final. If a component refers to a mutable object such as a List, that object can still change unless we make a defensive copy or use an immutable collection."

5. Can Records Extend Another Class?

No.

Every record implicitly extends:

</> Java



```
java.lang.Record
```

You cannot do:

</> Java



```
record Employee(...) extends Person { } // ❌
```

Records cannot extend another class.

However, they can implement interfaces.

6. Can Records Implement Interfaces?

Yes.

</> Java



```
interface Printable {  
    void print();  
}  
  
record Employee(Long id, String name) implements Printable {  
  
    @Override  
    public void print() {  
        System.out.println(name);  
    }  
}
```

This is completely valid.

Interview point

"Records cannot extend application classes, but they can implement one or more interfaces."

7. What are Sealed Classes?

Sealed classes were finalized in Java 17.

They allow a class or interface to explicitly control which classes are allowed to extend or implement it.

</> Java



```
sealed class Payment  
    permits CreditCardPayment, UPIPayment {  
}
```

Then:

</> Java



```
final class CreditCardPayment extends Payment {  
}
```

```
final class UPIPayment extends Payment {  
}
```

Only the permitted classes can extend `Payment` .

8. Why Use Sealed Classes?

The main purpose is **controlled inheritance**.

Normally:

</> **Java**



```
class Payment {  
}
```

Any class can extend it:

</> **Java**



```
class Something extends Payment {  
}
```

With sealed classes:

</> **Java**



```
sealed class Payment  
    permits CreditCardPayment, UPIPayment {  
}
```

You explicitly define the allowed hierarchy.

Real-world example

Suppose your payment system supports only:

Payment

├─ CreditCardPayment

├─ UPIPayment

└─ CashPayment



You don't want random implementations.

Sealed classes make the hierarchy explicit.

Benefits

- Controlled inheritance
- Better domain modeling
- Compiler can understand the complete hierarchy
- Works particularly well with pattern matching
- Makes code easier to reason about

9. Sealed Class vs Final Class

This is a common interview question.

`final`

`</> Java`

```
final class Payment {  
}
```



Nobody can extend it.

Payment

X no subclasses



`sealed`

`</> Java`

```
sealed class Payment  
    permits CardPayment, UPIPayment {  
}
```



Only specific classes can extend it.

Payment

├─ CardPayment

└─ UPIPayment



Simple answer

final means "no inheritance."

sealed means "restricted inheritance."

10. What is Pattern Matching?

Pattern matching allows you to combine:

1. Type checking
2. Type conversion/casting

into a more concise expression.

Traditional Java:

```
</> Java   
  
if (obj instanceof String) {  
    String str = (String) obj;  
    System.out.println(str.length());  
}
```

Pattern matching:

```
</> Java   
  
if (obj instanceof String str) {  
    System.out.println(str.length());  
}
```

The variable `str` is automatically created after the type check succeeds.

11. Pattern Matching with `instanceof`

Traditional:

```
</> Java   
  
if (employee instanceof Manager) {  
    Manager manager = (Manager) employee;
```

```
manager.manage();  
}
```

Pattern matching:

```
</> Java   
  
if (employee instanceof Manager manager) {  
    manager.manage();  
}
```

This reduces:

- explicit casting
- boilerplate
- possibility of casting mistakes

With condition

You can also do:

```
</> Java   
  
if (employee instanceof Manager manager  
    && manager.getTeamSize() > 10) {  
  
    System.out.println("Large team");  
}
```

12. Pattern Matching for switch

Modern Java allows pattern matching in switch.

For example:

```
</> Java   
  
static String getType(Object obj) {  
  
    return switch (obj) {  
  
        case Integer i -> "Integer: " + i;  
  
        case String s -> "String: " + s;  
  
    }  
}
```

```

        case Double d -> "Double: " + d;

        default -> "Unknown";
    };
}

```

Instead of:

```

</> Java

if (obj instanceof Integer) {
    ...
} else if (obj instanceof String) {
    ...
}

```

you can express different types directly through `switch`.

13. What are Virtual Threads?

Virtual threads were finalized in **Java 21** as part of Project Loom.

They are lightweight threads managed by the JVM rather than being directly mapped one-to-one to OS threads.

Traditional platform thread:

```

Java Thread
  ↓
OS Thread

```

Virtual thread:

```

Virtual Thread
  ↓
JVM scheduler
  ↓
Platform Thread
  ↓
OS Thread

```

The JVM can run a very large number of virtual threads using a much smaller number of platform threads.

Example:

 **Java**



```
Thread.startVirtualThread(() -> {  
    callExternalAPI();  
});
```

14. Platform Thread vs Virtual Thread

Platform Thread	Virtual Thread
Relatively expensive	Very lightweight
Backed by OS thread	Managed by JVM
Limited number practical	Very large number possible
Higher memory overhead	Much lower overhead
Good for CPU work	Excellent for blocking I/O
Traditional concurrency model	Modern lightweight concurrency

Think:

Platform threads:



1000 requests

↓

1000 threads ❌ expensive

Virtual threads:

1000 requests

↓

1000 virtual threads



↓
small number of platform threads

15. Why are Virtual Threads Useful for I/O-heavy Workloads?

Suppose your API does:

Request
↓
Database call
↓
External API
↓
Another service
↓
Response



A traditional thread might spend significant time **waiting** for I/O.

Virtual threads can be suspended while waiting, allowing the underlying platform thread to execute other work.

Conceptually:

Virtual Thread A
↓
Waiting for DB
↓
suspended

Platform Thread
↓
runs Virtual Thread B



When the DB response arrives:

Virtual Thread A
↓
resumes



This makes virtual threads particularly useful for applications with many concurrent blocking operations.

Examples:

- REST APIs
- Database calls
- HTTP calls
- File I/O
- Microservices

16. Should CPU-heavy Workloads Use Virtual Threads?

Generally, virtual threads don't magically make CPU-heavy work faster.

Example:

 `</> Java`

```
while (true) {  
    calculateSomething();  
}
```

If the task continuously consumes CPU, there isn't much blocking for virtual threads to eliminate.

CPU-heavy workloads are generally constrained by:

`Number of CPU cores` 

Virtual threads are primarily valuable when tasks spend significant time **waiting/blocking**.

Interview answer

"Virtual threads are optimized for high concurrency, especially I/O-bound workloads. For CPU-bound workloads, the limiting factor is CPU capacity, so virtual threads don't provide the same benefit."

17. Virtual Threads vs Traditional Thread Pools

Traditional approach:

</> Java



```
ExecutorService executor =  
    Executors.newFixedThreadPool(100);
```

You deliberately limit the number of threads.

Virtual-thread approach:

</> Java



```
ExecutorService executor =  
    Executors.newVirtualThreadPerTaskExecutor();
```

Each task can get its own virtual thread.

</> Java



```
try (ExecutorService executor =  
    Executors.newVirtualThreadPerTaskExecutor()) {  
  
    executor.submit(() -> callService());  
    executor.submit(() -> callDatabase());  
}
```

Important interview nuance

Don't say:

"Virtual threads replace all thread pools."

Better:

"Virtual threads reduce the need for manually sizing thread pools for blocking tasks. However, concurrency still needs to be controlled when downstream resources such as databases or external APIs have capacity limits."

For example, you could have 50,000 virtual threads but only a DB connection pool of 100 connections.

The bottleneck remains the database.

18. How does Spring Boot support Virtual Threads?

Spring Boot applications running on Java 21+ can enable virtual threads using:

```
</> properties
```



```
spring.threads.virtual.enabled=true
```

Spring Boot can then use virtual threads for supported task execution.

For example, applications using:

- `@Async`
- task execution
- scheduling

can benefit depending on the Spring Boot/Spring component and configuration.

Interview answer

"In Spring Boot, virtual threads can be enabled with `spring.threads.virtual.enabled=true` when running on a compatible Java version. Spring Boot integrates virtual threads into supported task-execution infrastructure."

19. What Problems Can Still Occur with Virtual Threads?

This is an important 5-year interview question.

Virtual threads solve the **thread scalability problem**, not every scalability problem.

1. Database connection pool

You could have:

```
50,000 virtual threads
```

```
↓
```

```
100 DB connections
```



Only 100 database operations can execute concurrently through that pool.

2. Downstream service overload

Suppose:

100,000 virtual threads
↓
External API



You could overwhelm the external service.

You still need:

- rate limiting
- concurrency limits
- timeouts
- circuit breakers

3. CPU saturation

Virtual threads don't create more CPUs.

8 CPU cores



still means roughly the same computational capacity.

4. Pinning

Certain operations can prevent a virtual thread from efficiently unmounting from its carrier thread, such as some uses of intrinsic locking/native operations.

Modern Java has improved this area considerably, but developers should still understand that **not every blocking scenario behaves identically**.

5. ThreadLocal usage

Virtual threads make it cheap to create many threads, but careless use of `ThreadLocal` can still consume memory or create undesirable context-management behavior.

Strong interview answer

"Virtual threads improve thread scalability, but they don't remove bottlenecks such as database connections, CPU, downstream service capacity, locks, memory, or application-level concurrency limits."

20. What is `var` ?

`var` was introduced in **Java 10**.

It allows **local variable type inference**.

Instead of:

```
</> Java
String name = "Manoj";
```



you can write:

```
</> Java
var name = "Manoj";
```



The compiler infers:

```
</> Java
String
```



Important:

Java is still statically typed.

`var` does **not** mean dynamically typed.

21. Where can `var` be used?

Primarily for **local variables**.

Local variable

</> Java



```
var name = "Java";
```

Loop variable

</> Java



```
for (var employee : employees) {  
    System.out.println(employee);  
}
```

Try-with-resources

</> Java



```
try (var stream = Files.lines(path)) {  
    // ...  
}
```

Local variable with initialization

</> Java



```
var employees = new ArrayList<Employee>();
```

22. Can var be Used for Class Fields?

No.

This is invalid:

</> Java



```
class Employee {  
  
    var name = "Manoj"; // ✗  
}
```

`var` is intended for local variable type inference, not fields.

Also invalid:

</> Java



```
var method() { } // ❌
```

You cannot use `var` as a method return type.

23. What are Text Blocks?

Text blocks were introduced in **Java 15**.

They make writing multi-line strings easier.

Traditional:

</> Java



```
String json = "{\n" +  
    "  \"name\": \"Manoj\", \n" +  
    "  \"age\": 29\n" +  
    "}";
```

Text block:

</> Java



```
String json = ""  
    {  
        "name": "Manoj",  
        "age": 29  
    }  
    "";
```

Useful for:

- JSON
- SQL
- HTML
- XML
- Multi-line messages

Example:

</> Java

```
String sql = ""
    SELECT id, name
    FROM employee
    WHERE department = 'IT'
    "";
```

Interview answer

"Text blocks provide a cleaner syntax for multi-line string literals and are especially useful for embedded SQL, JSON, XML, and HTML."

 Quick Java 17/21 Interview Revision

Feature	Java	Main Purpose
var	Java 10	Local variable type inference
Text Blocks	Java 15	Multi-line strings
Records	Java 16	Concise data carriers
Pattern matching instanceof	Java 16	Type check + cast
Sealed Classes	Java 17	Controlled inheritance
Pattern matching switch	Java 21	Type-based switch patterns
Virtual Threads	Java 21	Lightweight concurrency

The 5-year-experience summary

If the interviewer asks "What's new in modern Java that you have actually used?", a strong answer is:

"Modern Java introduced several features that reduce boilerplate and improve concurrency. Records simplify immutable DTO-style data carriers, sealed classes provide controlled inheritance, pattern matching simplifies type checks and switch

logic, text blocks improve multi-line strings, and virtual threads provide lightweight concurrency for high-concurrency I/O-bound applications. `var` provides local type inference while preserving Java's static typing."



What is serialization?

What is Serializable?

What is serialVersionUID?

What happens if serialVersionUID changes?

What is transient?

Can static variables be serialized?

Can final variables be serialized?

Serialization vs externalization.

Why is Java native serialization often avoided in modern systems?

JSON serialization vs Java serialization.

For a **5-year Java interview**, these are important because interviewers often go beyond definitions and ask about **security, compatibility, and production use cases**.

1. What is Serialization?

Serialization is the process of converting an object's state into a byte stream so that it can be:

- Stored in a file
- Sent over a network
- Cached
- Transferred between JVMs

</> Java



```
class Employee implements Serializable {  
    private Long id;  
    private String name;  
}
```

Then:

</> Java



```
ObjectOutputStream out =  
    new ObjectOutputStream(new FileOutputStream("employee.ser"));  
  
out.writeObject(employee);
```

The object is converted into bytes.

Java Object



Serialization



Byte Stream



File / Network / Cache



The reverse operation is **deserialization**:

Byte Stream



Deserialization



Java Object



2. What is Serializable ?

Serializable is a **marker interface**.

</> Java



```
public interface Serializable {  
}
```

It doesn't define methods.

If a class implements it:

</> Java



```
class Employee implements Serializable {  
    private Long id;  
    private String name;  
}
```

Java's serialization mechanism is allowed to serialize instances of that class.

Interview answer

"Serializable is a marker interface that tells Java that objects of the class can participate in the default Java serialization mechanism."

3. What is serialVersionUID?

serialVersionUID is a version identifier used during deserialization to verify that the serialized object and the current class are compatible.

Example:

</> Java



```
class Employee implements Serializable {  
  
    private static final long serialVersionUID = 1L;  
  
    private Long id;  
    private String name;  
}
```

Think of it as:

Serialized object



serialVersionUID = 1

Current Employee class



serialVersionUID = 1



Compatible 



If the values don't match:

Serialized object → 1
Current class → 2



Incompatible ❌

Usually you'll get:

java.io.InvalidClassException



4. What Happens if serialVersionUID Changes?

Suppose version 1:

</> Java



```
class Employee implements Serializable {  
  
    private static final long serialVersionUID = 1L;  
  
    private String name;  
}
```

An object is serialized.

Later you change:

</> Java



```
private static final long serialVersionUID = 2L;
```

When you try to deserialize an object created using version 1:

Serialized object
serialVersionUID = 1



Current class
serialVersionUID = 2



InvalidClassException ❌

Why?

Java assumes the class versions are incompatible.

Interview nuance

Changing the `serialVersionUID` intentionally can be used to indicate that old serialized data should no longer be considered compatible.

5. What is transient?

`transient` tells Java's default serialization mechanism:

Do not serialize this field.

Example:

```
</> Java   
  
class Employee implements Serializable {  
  
    private String name;  
  
    private transient String password;  
  
}
```

When serialized:

```
name      → serialized  
password → NOT serialized 
```

After deserialization, a transient field gets its default value.

For example:

```
</> Java   
  
String → null  
int    → 0  
boolean → false
```

Common use cases

Use `transient` for fields such as:

- Sensitive information
- Temporary/calculated data
- Non-serializable resources
- Caches
- Connections

Example:

 Java



```
private transient Connection connection;
```

A database connection should generally not be serialized.

6. Can Static Variables be Serialized?

No.

Static variables belong to the **class**, not to an individual object.

 Java



```
class Employee implements Serializable {  
  
    private String name;  
  
    private static String company = "ABC";  
}
```

`company` is not serialized as part of the object's state.

Important interview point

Static fields are effectively ignored by Java's default object serialization.

After deserialization, the static variable gets whatever value the **currently loaded class** has.

7. Can Final Variables be Serialized?

Yes, if they are instance fields and their types are serializable.

Example:

</> Java



```
class Employee implements Serializable {  
  
    private final Long id;  
    private final String name;  
  
    Employee(Long id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```

These fields can be serialized.

Important distinction

`final` does **not** mean "not serializable."

These concepts are unrelated:

```
final      → cannot reassign the field  
transient  → don't serialize the field
```



For example:

</> Java



```
private final String name;      // serialized  
private transient String token; // not serialized
```

8. Serialization vs Externalization

Both are mechanisms for converting an object into a byte representation, but they provide different levels of control.

Serialization

</> Java



```
class Employee implements Serializable {  
}
```

Java handles most of the serialization automatically.

Object



Java Serialization mechanism



Bytes



Externalization

The class implements:

</> Java



```
Externalizable
```

and explicitly defines:

</> Java



```
writeExternal()  
readExternal()
```

Example:

</> Java



```
class Employee implements Externalizable {  
  
    private Long id;  
    private String name;  
  
    @Override  
    public void writeExternal(ObjectOutput out)  
        throws IOException {  
  
        out.writeLong(id);  
        out.writeUTF(name);  
    }  
}
```

```
}

@Override
public void readExternal(ObjectInput in)
    throws IOException {

    id = in.readLong();
    name = in.readUTF();
}
}
```

You control exactly what gets written and read.

Comparison

Serialization	Externalization
Implements <code>Serializable</code>	Implements <code>Externalizable</code>
Mostly automatic	Explicit control
Less code	More code
Less control	More control
Easier	More complex
Can be less efficient	Can provide more control over representation

Interview answer

" `Serializable` provides default automatic serialization, whereas `Externalizable` gives the developer explicit control through `writeExternal()` and `readExternal()` ."

9. Why is Java Native Serialization Often Avoided in Modern Systems?

This is a very important production-level question.

Java native serialization has several problems.

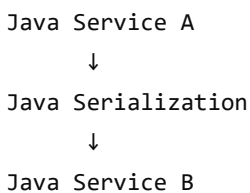
1. Security risks

Deserializing untrusted Java serialized data can be dangerous because deserialization can trigger object construction and potentially execute unintended behavior through gadget chains.

This is one of the biggest reasons modern systems avoid accepting arbitrary Java serialized data.

2. Tight coupling to Java

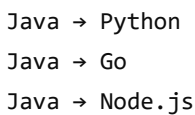
The serialized format is strongly tied to Java classes.



```
graph TD; A[Java Service A] --> B[Java Serialization]; B --> C[Java Service B];
```



This isn't ideal for:



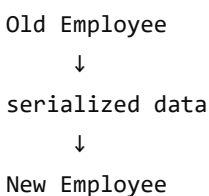
```
graph TD; A[Java] --> B[Python]; A --> C[Go]; A --> D[Node.js];
```



3. Version compatibility

Changes to classes can cause compatibility issues.

For example:



```
graph TD; A[Old Employee] --> B[serialized data]; B --> C[New Employee];
```



You need to carefully manage serialization compatibility.

4. Not ideal for microservices

Modern microservices generally prefer explicit, language-independent formats.

For example:

JSON
Protobuf
Avro



5. Performance and payload concerns

Java native serialization can produce payloads that aren't as efficient or convenient as modern serialization formats.

10. JSON Serialization vs Java Serialization

This is another very common interview question.

Java Serialization

</> Java



```
class Employee implements Serializable {  
}
```

Produces Java-specific serialized data.

Java Object
↓
Java Serialization
↓
Binary Java-specific format



JSON Serialization

Example:

</> Java



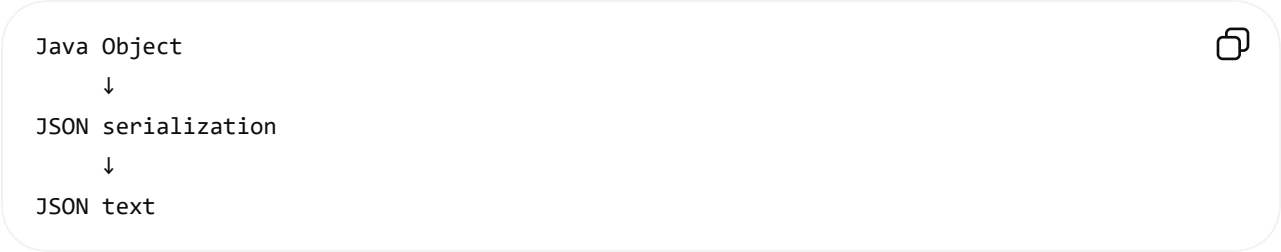
```
Employee employee =  
    new Employee(101L, "Manoj");
```


JSON:

</> JSON

```
{  
  "id": 101,  
  "name": "Manoj"  
}
```

Conceptually:



Comparison

Java Serialization	JSON
Java-specific	Language-independent
Binary	Text-based
Tightly coupled to Java classes	Loosely coupled
Security concerns with untrusted input	Generally easier to control/validate
Less suitable for public APIs	Very common for REST APIs
Can have compatibility challenges	Easier schema evolution
Mainly Java ecosystem	Java, Python, Go, JS, etc.

 Real REST API Example

Suppose you have:

</> Java



```
class Employee {  
    Long id;  
    String name;  
}
```

Your Spring Boot REST API returns:

</> Java



```
@GetMapping("/employees/{id}")  
public Employee getEmployee(@PathVariable Long id) {  
    return employeeService.getEmployee(id);  
}
```

Spring/Jackson can serialize it to:

</> JSON



```
{  
  "id": 101,  
  "name": "Manoj"  
}
```

The client doesn't need to know:

</> Java



```
class Employee implements Serializable
```

That's one reason JSON is much more appropriate for typical REST communication.

5-Year Interview Scenario

Interviewer:

"Would you use Java Serializable for communication between microservices?"

A strong answer:

"Generally no. For microservice communication, I would prefer a language-independent serialization format such as JSON for REST APIs or Protobuf/Avro for

high-performance or schema-driven communication. Java native serialization creates tighter coupling to Java classes and has security and compatibility concerns, especially when deserializing untrusted data."

Important Traps to Remember

transient VS static

 Java



```
private transient String password;
```

 Instance field, but excluded from serialization.

 Java



```
private static String company;
```

 Not part of serialized object state.

final VS transient

 Java



```
private final String name;
```

 Can be serialized.

 Java



```
private transient String password;
```

 Not serialized.

final does not prevent serialization.

serialVersionUID

Remember:

Same UID



Potentially compatible

Different UID



InvalidClassException



And the key production point:

Never deserialize untrusted Java native serialization data.

One-line revision

Question	Answer
Serialization	Object → byte stream
Deserialization	Byte stream → object
Serializable	Marker interface
serialVersionUID	Serialization compatibility/version identifier
UID changes	Can cause <code>InvalidClassException</code>
transient	Excludes instance field from default serialization
static field	Not serialized as object state
final field	Can be serialized
Externalization	Manual serialization control
Java serialization	Java-specific and has security/compatibility concerns
JSON	Language-independent, common for APIs



ChatGPT can make mistakes. Check important info.